

## CO1-AT3-Coding Challenge

View Only

[← Back](#)

### QUESTIONS

**Q1**Smart University Payroll  
Management System  
25 marks**Q2**Smart Vehicle Insurance Claim  
System  
25 marks

2/2 answered

### Q2 Smart Vehicle Insurance Claim System

25 marks

**Problem Statement** An insurance company processes claims for three categories of vehicles: Car, Bike, and Truck. The claim amount depends on the vehicle type. Requirements Superclass: Vehicle Private members: vehicleNumber ownerName insuredAmount Subclasses: Car Bike Truck Each subclass overrides calculateClaimAmount() using different formulas. The application should: Read details for N vehicles Store them using an array of superclass references Calculate claim amounts using runtime polymorphism Search by vehicle number Display the vehicle with the highest claim amount

**Question 3 (Very Hard) Online Shopping Order Processing System (50 Marks)**

**Problem Statement** An e-commerce company sells products in three categories: Electronics Clothing Grocery The billing process varies for each category. Requirements Superclass: Product Private members: productId productName price quantity Subclasses: Electronics Clothing Grocery Each subclass overrides calculateBill(). The program should: Store products in an array of superclass references Calculate the final bill using runtime polymorphism Search products by ID Display the most expensive order Compute total revenue

Claim Formula (Recommended)

Car

$$\text{Claim} = \text{insuredAmount} \times 0.80$$

Bike

Claim = insuredAmount × 0.60

Truck

Claim = insuredAmount × 0.90

Test Case 1

Input

3

Car

TN10AB1234

Ramesh

800000

Bike

TN09XY5678

Suresh

100000

Truck

TN11TR9999

Logan

1500000

Search Vehicle:

TN09XY5678

Expected Output

Vehicle : TN10AB1234

Claim Amount : 640000.0

Vehicle : TN09XY5678

Claim Amount : 60000.0

Vehicle : TN11TR9999

Claim Amount : 1350000.0

Highest Claim Vehicle

TN11TR9999

1350000.0

Vehicle Found

Owner : Suresh

Claim Amount : 60000.0

### Your Code

```

1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int n = Integer.parseInt(sc.nextLine().trim());
8         Vehicle[] v = new Vehicle[n];
9
10        for (int i = 0; i < n; i++) {
11            String type = sc.nextLine().trim();
12            while (type.isEmpty()) type = sc.nextLine().trim();
13
14            String number = sc.nextLine().trim();
15            String owner = sc.nextLine().trim();
16            double amount = Double.parseDouble(sc.nextLine().trim());

```

### Input

3  
Car  
TN10AB1234  
Ramesh

### Output

Vehicle : TN10AB1234  
Claim Amount :  
640000.0  
  
Vehicle : TN09XY5678  
Claim Amount : 60000.0

```
17
18         if (type.equalsIgnoreCase("Car"))
19             v[i] = new Car(number, owner, amount);
20         else if (type.equalsIgnoreCase("Bike"))
21             v[i] = new Bike(number, owner, amount);
22         else if (type.equalsIgnoreCase("Truck"))
23             v[i] = new Truck(number, owner, amount);
24     }
25
26     String temp = sc.nextLine().trim();
27     while (temp.isEmpty()) temp = sc.nextLine().trim();
28
29     String search;
30     if (temp.equalsIgnoreCase("Search Vehicle:"))
31         search = sc.nextLine().trim();
32     else
33         search = temp;
34
35     Vehicle highest = v[0];
36     for (Vehicle x : v) {
37         System.out.println("Vehicle : " + x.getVehicleNumber());
38         System.out.println("Claim Amount : " + x.calculateClaimAmount());
39         System.out.println();
40
41         if (x.calculateClaimAmount() > highest.calculateClaimAmount())
42             highest = x;
43     }
44
45
46     System.out.println("Highest Claim Vehicle")
47     System.out.println(highest.getVehicleNumber());
48     System.out.println(highest.calculateClaimAmount());
49     System.out.println();
```

```
50
51     boolean found = false;
52     for (Vehicle x : v) {
53         if (x.getVehicleNumber().equals(search))
54             found = true;
55         System.out.println("Vehicle Found")
56         System.out.println("Owner : " + x.g
57         System.out.println("Claim Amount :
58         break;
59     }
60 }
61
62 if (!found) {
63     System.out.println("Vehicle not Found")
64 }
65
66 sc.close();
67 }
68 }
69
70 class Vehicle {
71     private String vehicleNumber;
72     private String ownerName;
73     private double insuredAmount;
74
75     Vehicle(String vehicleNumber, String ownerName,
76         this.vehicleNumber = vehicleNumber;
77         this.ownerName = ownerName;
78         this.insuredAmount = insuredAmount;
79     }
80
81     public String getVehicleNumber() {
82         return vehicleNumber;
```

```
83     }
84
85     public String getOwnerName() {
86         return ownerName;
87     }
88
89     public double getInsuredAmount() {
90         return insuredAmount;
91     }
92
93     public double calculateClaimAmount() {
94         return insuredAmount;
95     }
96 }
97
98 class Car extends Vehicle {
99     Car(String v, String o, double a) {
100         super(v, o, a);
101     }
102
103     public double calculateClaimAmount() {
104         return getInsuredAmount() * 0.80;
105     }
106 }
107
108 class Bike extends Vehicle {
109     Bike(String v, String o, double a) {
110         super(v, o, a);
111     }
112
113     public double calculateClaimAmount() {
114         return getInsuredAmount() * 0.60;
115     }
```

```
116 }
117
118 class Truck extends Vehicle {
119     Truck(String v, String o, double a) {
120         super(v, o, a);
121     }
122
123     public double calculateClaimAmount() {
124         return getInsuredAmount() * 0.90;
125     }
126 }
```

 Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.